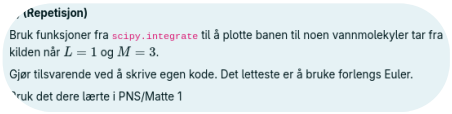
Log in

You said:


(Repetisjon)
Bruk funksjoner fra `scipy.integrate` til å plotte banen til noen vannmolekyler tar fra kilden når $L = 1$ og $M = 3$.
Gjør tilsvarende ved å skrive egen kode. Det letteste er å bruke forlengs Euler.
Bruk det dere lærte i PNS/Matte 1

Hvordan gjør jeg denne? Mener de her "Eulers metode" når de sier forlengs Euler?

ChatGPT said:

Ja! 😊 **Forlengs Euler er det samme som Eulers metode (eksplisitt Euler-metode).**

Oppgaven ber deg altså om å lage en egen kode som bruker Eulers metode til å plotte banen til noen vannmolekyler fra kilde N til $L = 1$ og $M = 3$.

Jeg kan forklare hvordan du gjør det steg for steg.

1. Hva er forlengs Euler?

Forlengs Euler er en numerisk metode som brukes til å løse differensiallikninger.

Hvis vi har

$$\frac{dy}{dt} = f(t, y)$$

bruker vi følgende formel:

$$y_{n+1} = y_n + h \cdot f(t_n, y_n)$$

Der:

- h er steglengden.
- y_n er verdien vi har nå.
- $f(t_n, y_n)$ er den deriverte (endringen) akkurat nå.
- y_{n+1} er neste verdi.

Vi bruker altså stigningstallet der vi er nå for å regne ut neste punkt.

Ask ChatGPT

Bruke `scipy.integrate` til å plotte banen først.

6. Lage din egen kode som gjør det samme ved hjelp av forlengs Euler.
7. Sammenligne resultatene fra de to metodene.

Et generelt eksempel på hvordan du kan programmere forlengs Euler i Python:

python

```python
import numpy as np
import matplotlib.pyplot as plt

# Steglengde
h = 0.01

# Startverdier
t = np.arange(0, 10, h)

x = np.zeros(len(t))
y = np.zeros(len(t))

x[0] = 0
y[0] = 0

# Forlengs Euler
for i in range(len(t)-1):
    x[i+1] = x[i] + h * vx(x[i], y[i])
    y[i+1] = y[i] + h * vy(x[i], y[i])

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Her er `vx` og `vy` funksjoner som beskriver hastigheten i henholdsvis x- og y-retning. Disse må erstattes med uttrykkene fra oppgaven.

3. Jeg trenger én ting til

For å vise deg nøyaktig hvordan du løser akkurat denne oppgaven, trenger jeg å se

Log in

Ask ChatGPT

Kan du sende et bilde av del a) også?

Log in

Da kan vi skrive hele Euler-koden sammen, med de riktige ligningene, og plotte vannmolekylenes baner.

You said:

Hva betyr denne erroren?:

ChatGPT said:

Kan du sende et bilde av feilmeldingen eller lime inn hele error-teksten her? 😊

Da kan jeg forklare:

- Hva feilen betyr.
- Hvorfor den oppstår.
- Hvordan du fikser den i Python-koden din.

You said:

Exception has occurred: ValueError
setting an array element with a sequence.
File "/home/arntzen/Desktop/IMAT301x/IMAT3012/oving5.py", line 91, in <module>
x[i+1] = x[i] + h * vx(x[i], y[i])
~^^^^^
ValueError: setting an array element with a sequence.

ChatGPT said:

Denne feilen betyr at Python prøver å sette inn **en liste eller et array (flere verdier)** i én enkelt plass i et array som bare forventer ett tall (`float` eller `int`).

Feilen oppstår på denne linjen:

python

```
x[i+1] = x[i] + h * vx(x[i], y[i])
```

Ask ChatGPT

Eksempel:

Log in

python

```
def vx(x, y):
    return [2, 3]
```

Da prøver Python å gjøre dette:

python

```
x[i+1] = 0.5 + 0.01 * [2, 3]
```

Men `x[i+1]` kan bare inneholde ett tall.

2. Hvordan finner du feilen?

Prøv å skrive dette rett før for-løkken:

python

```
print(vx(x[0], y[0]))
print(type(vx(x[0], y[0])))
```

Hvis du får ut en liste eller et array med flere verdier, har vi funnet problemet.

3. Hva trenger jeg for å hjelpe deg videre?

Kan du sende meg disse tre delene av koden?

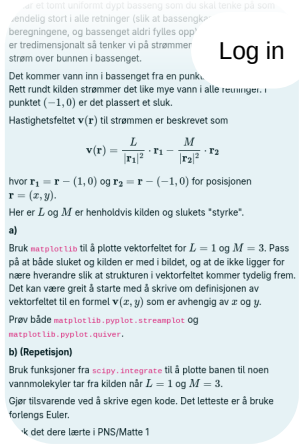
11. Funksjonen `vx()`.
12. Funksjonen `vy()`.
13. Der du definerer `x`, `y` og startverdiene deres.

Da kan jeg vise deg nøyaktig hva som må endres, slik at forlengs Euler fungerer som den skal. 😊

You said:

Ask ChatGPT

Log in



...ar et tomt uniformt dypt basseng som du skal tenke på som ...endelig stort i alle retninger (slik at bassengkan... beregningene, og bassenget aldri fylles opp... er tredimensjonalt så tenker vi på strømmer... strøm over bunnen i bassenget.

Det kommer vann inn i bassenget fra en punk... Rett rundt kilden strømmer det like mye vann i alle retninger. I punktet $(-1, 0)$ er det plassert et sluk.

Hastighetsfeltet $\mathbf{v}(\mathbf{r})$ til strømmen er beskrevet som

$$\mathbf{v}(\mathbf{r}) = \frac{L}{|\mathbf{r}_1|^2} \cdot \mathbf{r}_1 - \frac{M}{|\mathbf{r}_2|^2} \cdot \mathbf{r}_2$$

hvor $\mathbf{r}_1 = \mathbf{r} - (1, 0)$ og $\mathbf{r}_2 = \mathbf{r} - (-1, 0)$ for posisjonen $\mathbf{r} = (x, y)$.

Her er L og M er henholdvis kilden og slukets "styrke".

a)

Bruk `matplotlib` til å plotte vektorfeltet for $L = 1$ og $M = 3$. Pass på at både sluket og kilden er med i bildet, og at de ikke ligger for nære hverandre slik at strukturen i vektorfeltet kommer tydelig frem. Det kan være greit å starte med å skrive om definisjonen av vektorfeltet til en formel $\mathbf{v}(x, y)$ som er avhengig av x og y.

Prøv både `matplotlib.pyplot.streamplot` og `matplotlib.pyplot.quiver`.

b) (Repetisjon)

Bruk funksjoner fra `scipy.integrate` til å plotte banen til noen vannmolekyler tar fra kilden når $L = 1$ og $M = 3$.

Gjør tilsvarende ved å skrive egen kode. Det letteste er å bruke forlengs Euler.

...k det dere lærte i PNS/Matte 1

Her er bilde av hele oppgaven og her er koden min så langt for begge oppgavene:

```
# ============================================
#                 Oppgave 2
# ============================================
# a)
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-2, 2, 100)
y = np.linspace(-2, 2, 100)
X, Y = np.meshgrid(x, y)

L = 1
M = 3

r1_sq = (X - 1)**2 + Y**2
r2_sq = (X + 1)**2 + Y**2

U = L*(X - 1)/r1_sq - M*(X + 1)/r2_sq
V = L*Y/r1_sq - M*Y/r2_sq

# plt.figure(figsize=(8, 6))
# plt.streamplot(X, Y, U, V, density=2)
# # plt.quiver(X, Y, U, V, angles='xy', scale_units='xy', scale=10)
# plt.plot(1, 0, 'ro', label='Kilde')
# plt.plot(-1, 0, 'bo', label='Sluk')
# plt.xlabel('x')
# plt.ylabel('y')
# plt.legend()
```

Ask ChatGPT

Log in

```
# b)
from scipy import integrate

h = 0.01
t = np.arange(-2, 2, h)
x = np.zeros(len(t))
y = np.zeros(len(t))

r1_sq_s = (x - 1)**2 + y**2
r2_sq_s = (x + 1)**2 + y**2

def vx(x, y):
    return L*(x - 1)/r1_sq_s - M*(x + 1)/r2_sq_s

def vy(x, y):
    return L*y/r1_sq_s - M*y/r2_sq_s

print(type(vx(x[0], y[0])))
print(type(vy(x[0], y[0])))

for i in range(len(t) - 1):
    x[i+1] = x[i] + h * vx(x[i], y[i])
    y[i+1] = y[i] + h * vy(x[i], y[i])

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

ChatGPT said:

Worked for 12s

Nå ser jeg feilen! 😊 Du er faktisk veldig nær å få dette til. Problemet ligger i hvordan du har definert `r1_sq_s` og `r2_sq_s`.

Det er en ganske vanlig feil når man går fra å bruke `numpy`-arrayer til å bruke vanlige funksjoner i Eulers metode.

1. Hvorfor får du feilmeldingen?

Du har skrevet:

Ask ChatGPT

Log in

```
r1_sq_s = (x - 1)**2 + y**2
r2_sq_s = (x + 1)**2 + y**2
```

Problemet er at `x` og `y` her er hele arrayer med 400 elementer, ikke bare én enkelt x- og y-verdi.

Dermed blir også `r1_sq_s` og `r2_sq_s` arrayer.

Når du senere skriver:

python

```
def vx(x, y):
    return L*(x - 1)/r1_sq_s - M*(x + 1)/r2_sq_s
```

returnerer funksjonen en hel array i stedet for ett tall.

Men `x[i+1]` skal bare inneholde én enkelt verdi. Derfor får du:

```
ValueError: setting an array element with a sequence
```

2. Slik fikser du det

Du må flytte beregningen av `r1_sq` og `r2_sq` inn i funksjonene `vx()` og `vy()`.

Da beregnes avstanden på nytt for hvert punkt du beveger deg til.

Slik skal funksjonene se ut:

python

```
def vx(x, y):
    r1_sq = (x - 1)**2 + y**2
    r2_sq = (x + 1)**2 + y**2

    return L*(x - 1)/r1_sq - M*(x + 1)/r2_sq


def vy(x, y):
```

Ask ChatGPT

```
    return L*y/r1_sq - M*y/r2_sq
```

Log in

Nå returnerer begge funksjonene én enkelt hastighetskomponent, slik vi ønsker.

3. En annen ting du må endre

Du har også dette:

python

```
x = np.zeros(len(t))
y = np.zeros(len(t))
```

Dette betyr at du starter vannmolekylet i punktet $(0, 0)$.

Men oppgaven sier at vannmolekylene kommer fra kilden i punktet $(1, 0)$.

Du må derfor starte molekylene like ved kilden. Grunnen til at vi ikke starter akkurat i $(1, 0)$, er at hastighetsfeltet ikke er definert der (vi får divisjon på null).

4. Forslag til komplett Euler-kode

Her er hvordan jeg ville skrevet del b med forlengs Euler. Jeg lar flere vannmolekyler starte rundt kilden:

python

```
# b) Forlengs Euler

L = 1
M = 3

h = 0.0001

def vx(x, y):
    r1_sq = (x - 1)**2 + y**2
    r2_sq = (x + 1)**2 + y**2
    return L*(x-1)/r1_sq - M*(x+1)/r2_sq
```

Ask ChatGPT

```
    r2_sq = (x + 1)**2 + y**2
    return L*y/r1_sq - M*y/r2_sq

plt.figure()

# Starter flere molekyler rundt kilden
for theta in np.linspace(0, 2*np.pi, 12):

    x = 1 + 0.05*np.cos(theta)
    y = 0 + 0.05*np.sin(theta)

    x_bane = [x]
    y_bane = [y]

    for i in range(100000):

        # Forlengs Euler
        x_ny = x + h*vx(x, y)
        y_ny = y + h*vy(x, y)

        x = x_ny
        y = y_ny

        x_bane.append(x)
        y_bane.append(y)

        # Stopper når molekylet kommer nær sluket
        if np.sqrt((x+1)**2 + y**2) < 0.05:
            break

        # Stopper hvis molekylet forlater området
        if abs(x) > 3 or abs(y) > 3:
            break

    plt.plot(x_bane, y_bane)

plt.plot(1, 0, 'ro', label='Kilde')
plt.plot(-1, 0, 'bo', label='Sluk')
```

Log in

Ask ChatGPT

```
plt.axis('equal')
plt.legend()
plt.grid()
plt.show()
```

Log in

5. Hva skjer egentlig i Euler-metoden her?

Det viktigste er disse to linjene:

python

```
x_ny = x + h*vx(x, y)
y_ny = y + h*vy(x, y)
```

De tilsvarer akkurat formlene for forlengs Euler:

$$x_{n+1} = x_n + h \cdot v_x(x_n, y_n)$$

$$y_{n+1} = y_n + h \cdot v_y(x_n, y_n)$$

Vi bruker altså hastigheten i det nåværende punktet til å finne neste punkt. Så gjentar vi dette mange ganger for å få banen til vannmolekylet.

Et siste tips: Du har allerede gjort mye riktig i del a! Det er faktisk de samme uttrykkene for hastigheten du bruker i begge delene. Forskjellen er bare at i del a beregner du hastigheten for mange punkter samtidig med NumPy, mens i del b beregner du hastigheten punkt for punkt med Euler-metoden.

You said:

Må jeg introdusere sirkelkoordinater for x og y her?

ChatGPT said:

Nei, du trenger ikke å introdusere sirkelkoordinater (polarkoordinater) for å løse denne oppgaven. Du kan fint bruke vanlige kartesiske koordinater (x, y), slik du allerede gjør.

Ask ChatGPT

Du har hastighetsfeltet:

$$\mathbf{v}(x,y) = L\frac{\mathbf{r}_1}{|\mathbf{r}_1|^2} - M\frac{\mathbf{r}_2}{|\mathbf{r}_2|^2}$$

Dette deler vi opp i x- og y-retning:

$$v_x = L\frac{x-1}{(x-1)^2+y^2} - M\frac{x+1}{(x+1)^2+y^2}$$

$$v_y = L\frac{y}{(x-1)^2+y^2} - M\frac{y}{(x+1)^2+y^2}$$

Disse bruker du direkte i Euler-metoden.

2. Når kan sirkelkoordinater være nyttige?

I koden jeg viste tidligere, brukte vi:

python

```
for theta in np.linspace(0, 2*np.pi, 12):

    x = 1 + 0.05*np.cos(theta)
    y = 0.05*np.sin(theta)
```

Her bruker vi sirkelkoordinater til å plassere startpunktene til vannmolekylene på en liten sirkel rundt kilden.

Dette er bare en praktisk måte å velge startpunkter på.

Matematisk betyr det:

$$x = 1 + r\cos(\theta)$$

$$y = r\sin(\theta)$$

Så hva anbefaler jeg?

Log in

- **Selve Euler-metoden:** Bruk kartesiske koordinater x og y.
- **Startpunktene:** Du kan bruke sirkelkoordinater for å plassere molekylene rundt kilden.
- **Beregning av hastighet:** Bruk uttrykkene for v_x og v_y over.

Du trenger altså ikke gjøre oppgaven om til et polarkoordinatsystem. Det vil faktisk gjøre koden unødvendig komplisert. 😊

You said:

Hvordan blir det med kartesiske koordinater da?

ChatGPT said:

Da bruker du bare vanlige x- og y-koordinater til å bestemme hvor vannmolekylene starter. Du trenger ikke bruke vinkler (θ) eller sinus og cosinus.

1. Velg startpunkter

Kilden ligger i $(1, 0)$.

Du kan for eksempel starte vannmolekylene på forskjellige punkter like til venstre for kilden:

python

```
start_x = [0.9, 0.9, 0.9, 0.9, 0.9]
start_y = [-0.2, -0.1, 0, 0.1, 0.2]
```

Her starter alle molekylene nær kilden, men på litt forskjellige steder.

2. Bruk Euler-metoden

Da kan du skrive:

python

```
plt.figure()

for j in range(len(start_x)):
```

Ask ChatGPT

Log in

```
    x_bane = [x]
    y_bane = [y]

    for i in range(100000):

        x_ny = x + h*vx(x, y)
        y_ny = y + h*vy(x, y)

        x = x_ny
        y = y_ny

        x_bane.append(x)
        y_bane.append(y)

        if np.sqrt((x+1)**2 + y**2) < 0.05:
            break

    plt.plot(x_bane, y_bane)

plt.plot(1, 0, 'ro', label='Kilde')
plt.plot(-1, 0, 'bo', label='Sluk')

plt.xlabel('x')
plt.ylabel('y')
plt.axis('equal')
plt.legend()
plt.grid()
plt.show()
```

3. Hva er forskjellen?

Kartesiske koordinater	Sirkelkoordinater
x og y	r og θ
Bestemmer punkter direkte	Bestemmer punkter med avstand og vinkel
Enkelt å bruke i Euler	Praktisk for å plassere punkter rundt en sirkel

Ask ChatGPT

Husk at Euler-metoden fortsatt bruker de samme oppdateringsformlene for x og y, uans… …dan du velger startpunktene! 😊

Log in

You said:

hvordan kan vi gjøre dette med scipy.integrate?

ChatGPT said:

Her kan vi bruke `scipy.integrate.solve_ivp()` til å løse differensiallikningene numerisk. Dette er egentlig en enklere måte å gjøre del b på, siden SciPy gjør selve integreringen for oss.

Vi bruker fortsatt kartesiske koordinater x og y.

1. Først definerer vi hastighetsfeltet

Fra oppgaven har vi:

$$v_x(x,y) = \frac{L(x-1)}{(x-1)^2+y^2} - \frac{M(x+1)}{(x+1)^2+y^2}$$

$$v_y(x,y) = \frac{Ly}{(x-1)^2+y^2} - \frac{My}{(x+1)^2+y^2}$$

Vi setter $L = 1$ og $M = 3$.

2. Lag en funksjon som beskriver bevegelsen

`solve_ivp()` trenger en funksjon som returnerer de deriverte av posisjonen, altså hastighetene.

python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

L = 1
M = 3
```

Ask ChatGPT

```
    x, y = r

    r1_sq = (x - 1)**2 + y**2
    r2_sq = (x + 1)**2 + y**2

    vx = L*(x - 1)/r1_sq - M*(x + 1)/r2_sq
    vy = L*y/r1_sq - M*y/r2_sq

    return [vx, vy]
```

Log in

Legg merke til at:

- `r` inneholder posisjonen $[x, y]$.
- `vx` er hastigheten i x-retning.
- `vy` er hastigheten i y-retning.
- Vi returnerer `[vx, vy]`.

3. Bruk solve_ivp til å plotte banene

Vi starter flere vannmolekyler i forskjellige punkter nær kilden.

python

```
plt.figure()

start_x = [0.9, 0.9, 0.9, 0.9, 0.9]
start_y = [-0.2, -0.1, 0, 0.1, 0.2]

for i in range(len(start_x)):

    r0 = [start_x[i], start_y[i]]

    sol = solve_ivp(
        v,
        [0, 10],
        r0,
        max_step=0.01
    )
```

Ask ChatGPT

```
plt.plot(-1, 0, 'bo', label='Sluk')
plt.xlabel('x')
plt.ylabel('y')
plt.axis('equal')
plt.legend()
plt.grid()
plt.show()
```

4. Hva gjør solve_ivp?

Det viktigste er denne delen:

python

```
sol = solve_ivp(v, [0, 10], r0, max_step=0.01)
```

Argument	Betydning
`v`	Funksjonen som beskriver hastigheten
`[0, 10]`	Tidsintervallet vi integrerer over
`r0`	Startposisjonen til vannmolekylet
`max_step`	Maksimal steglengde

Resultatet lagres i `sol`.

- `sol.y[0]` inneholder alle x-posisjonene.
- `sol.y[1]` inneholder alle y-posisjonene.
- `sol.t` inneholder tidspunktene.

5. Sammenhengen med forlengs Euler

Dette er egentlig det oppgaven vil at du skal sammenligne:

`scipy.integrate`	**Egen Euler-kode**
`solve_ivp()` utfører integreringen	Du utfører integreringen selv
Velger steglengde automatisk (med mindre annet angis)	Du bestemmer steglengden h

Log in

Ask ChatGPT

Viktig: `solve_ivp()` bruker som standard Runge-Kutta-metoden RK45, ikke forlengs Euler. Det er nettopp derfor det er interessant å sammenligne resultatene!

Da har du i praksis løst begge delene av oppgave b. 😊